Hand Gesture Robot Control

Project Documentation — Structured English Translation

Wireless control of a four-wheel Differential Drive robot using hand-gesture recognition through a camera, image processing with MediaPipe in Python, and command transmission over Wi-Fi using the UDP protocol to an ESP32 board. The ESP32 controls the robot motors through an L298N motor driver.

1. Introduction and Overall Architecture

The system consists of three main layers:

- Detection layer (Python + MediaPipe): Reads camera frames, extracts the 21 hand-skeleton landmarks, and converts gestures into Command objects.
- Output layer (Sinks): Each Command is sent simultaneously to multiple destinations: the console, a JSON log file, a browser live-display WebSocket, and UDP communication with the robot.
- Receiving layer (ESP32): Continuously listens on a UDP port, maps received packets to movement commands, and controls the L298N pins.

Overall data flow: Laptop Camera → Python / MediaPipe Hand Detection → UDP over Wi-Fi → ESP32 → L298N → Robot Motors.

2. Required Components

Hardware

- ESP32-WROOM-32U board, 32-pin
- L298N motor driver
- 4 DC motors + robot chassis + 4 wheels
- Main robot power supply: two 3.7 V lithium-ion 18650 batteries with a dual 18650 battery holder, connected in series
- LM2596 DC-DC buck converter module for reducing the battery voltage to a stable 5 V supply for the ESP32
- Mini breadboard

Software

- Python 3.10 or later
- Arduino IDE for uploading code to the ESP32
- Python libraries: opencv-python, mediapipe, pyyaml, flask, websockets

3. Installing and Setting Up the Python Software

1) Download the project repository and enter its directory:

```
cd hand_gesture_control
```

2) Install the required libraries:

```
pip install -r requirements.txt
```

3) The hand-detection model file (hand_landmarker.task, approximately 10 MB) must be located in the models/ directory. If automatic downloading fails, for example with a 403 Forbidden error, download it manually:

```
mkdir models
Invoke-WebRequest -Uri "https://storage.googleapis.com/mediapipe-
models/hand_landmarker/hand_landmarker/float16/1/hand_landmarker.task" -OutFile
"models\hand_landmarker.task"
```

4) Run the program:

```
python main.py
```

After startup, a camera window showing the hand skeleton will appear. The web live display will be available at http://127.0.0.1:8000.

4. Hardware Wiring

ESP32-to-L298N connections:

ESP32 Pin (GPIO)	L298N Pin
13	IN1
12	IN2
14	IN3
27	IN4
VIN	VCC
GND	GND

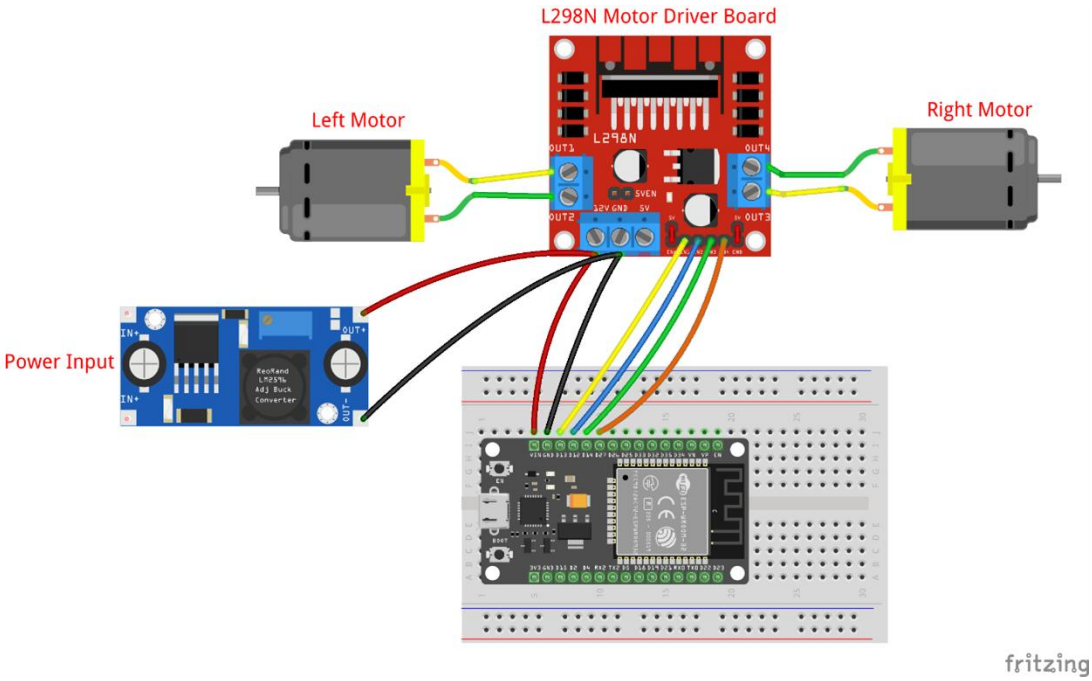

Figure: Complete wiring overview of the ESP32, L298N driver, LM2596 power module, and left/right motors.

Note: In this configuration, the ENA and ENB pins on the L298N driver must remain directly connected to the supply voltage or connected through the board's default jumpers so that the motors are always enabled at maximum speed. This code does not implement variable-speed (PWM) control.

Power Supply (LM2596 Module)

In this project, the robot operates at 5 V. To provide a stable voltage, two 3.7 V lithium-ion batteries connected in series are fed into an LM2596 DC-DC buck converter. The module precisely reduces the battery input voltage to an adjustable 5 V output.

- Connect LM2596 IN+ and IN− to the two terminals of the main battery pack.
- Before connecting the output, adjust OUT+ and OUT− to exactly 5 V using a multimeter and the module's potentiometer.
- Connect the module's 5 V output to the ESP32 VIN and GND pins and also to the L298N logic power input.
- Using an LM2596 instead of a linear regulator is more suitable for battery-powered operation because of its higher efficiency and lower heat generation.
- The ESP32 can be powered from the LM2596 5 V output or, during programming, through a Micro-USB cable.

- The motors are powered from the LM2596 through the L298N.
- The grounds (GND) of the ESP32, L298N, battery, and motor power system must be common.

5. ESP32 Code

The complete code below should be entered in Arduino IDE and uploaded to the ESP32 board. Before uploading, replace the ssid and password values with the actual Wi-Fi credentials.

First of all, the code accompanying this documentation is located in a folder named "wifi"; the file inside has the .ino extension, meaning you need to open it using the Arduino software.

Features of this code:

- Static IP configuration so the board's address does not change through the router.
- Wi-Fi sleep disabled with WiFi.setSleep(false) to prevent packet-reception delays or interruptions.
- Automatic reconnection if Wi-Fi is disconnected.
- Listening on UDP port 4210 and mapping the received code to the corresponding movement.

6. Mapping Gestures to Commands

The detection system identifies each gesture using a unique numeric code (Command.code). The following table shows the mapping used in this project:

Code(s)	Robot Movement
9, 10, 11	Forward
15, 16	Backward
6	Stop
0, 1, 2, 3	Left
12, 13, 14	Right

7. UDP Sink Code (Python Side)

The file core/output/udp_sink.py is responsible for sending the command code to the ESP32. It includes a stability filter that prevents noisy and momentary codes from being transmitted.

```
import socket
from .base_sink import OutputSink
```

```
from ..command import Command


class UDPSink(OutputSink):
    def __init__(self, ip="192.168.1.150", port=4210, categories=None,
stability_count=2):
        super().__init__(categories)
        self.addr = (ip, port)
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self._last_sent_code = None
        self._candidate_code = None
        self._candidate_count = 0
        self._stability_count = stability_count

    def emit(self, command: Command):
        if not self.should_emit(command):
            return
        code = command.code
        if code == self._candidate_code:
            self._candidate_count += 1
        else:
            self._candidate_code = code
            self._candidate_count = 1
        if self._candidate_count < self._stability_count:
            return
        if code == self._last_sent_code:
            return
        self._last_sent_code = code
        try:
            self.sock.sendto(str(code).encode("utf-8"), self.addr)
            print(f"[UDPSink] ارسال شد: {code}")
        except OSError as e:
            print(f"[UDPSink] خطا: {e}")

    def close(self):
        self.sock.close()
```

8. Complete System Execution

- Power on the ESP32 and wait for it to connect to Wi-Fi. Verify its status through Serial Monitor at 115200 baud.
- Make sure the IP configured in UDPSink in main.py matches the ESP32's static IP.
- Run the Python program with: python main.py
- Place your hand in front of the camera and perform the defined gestures.

9. Configurable Settings

Parameter	Description
ip	Static IP address of the ESP32 board
port	UDP port (default: 4210)
categories	Only commands in these categories are sent, e.g. ["gesture", "movement"]
stability_count	Number of consecutive frames required to accept a code as stable; a higher value means greater stability but slower response

10. Troubleshooting

Problem	Possible Cause / Solution
Code is printed but does not reach the ESP32	Incorrect IP in main.py — use the static IP.
Only a few packets are received and then communication stops	ESP32 Wi-Fi sleep mode — add WiFi.setSleep(false).
The code keeps changing even when the hand is steady	Natural detector noise — increase stability_count.
A new number is sent every frame	High-volume category is not filtered — restrict categories.
ModuleNotFoundError	Incorrect import path — verify the actual project directory path.
Model download error (HTTP 403)	Download the model manually using Invoke-WebRequest.
ESP32 resets after the motors are connected	Voltage drop — separate the motor power supply from the ESP32.

11. Future Developments

- Improve hand-detection accuracy: fine-tune MediaPipe model confidence thresholds, improve lighting in the test environment, and add smoothing to hand-skeleton landmarks to reduce jitter and incorrect gesture recognition.
- Add variable-speed control based on the intensity of hand movement by restoring ENA/ENB control and PWM.
- Add a distance sensor for automatic collision avoidance.
- Support multiple robots simultaneously by including a device identifier in the UDP message.